Group Practical Assignment
IoT Programming
SWE30011

Smart Home System

Abdul Hamid Mahi, 103521410
Jason Stanley, 102395384
Valentin Gurzau, 103062540

TABLE OF CONTENTS

Introduction

Smart homes are becoming increasingly popular as the internet gets faster and more accessible. A smart house is a home setting in which internet-connected appliances and equipment may be managed remotely through a networked device or through a single central point: a smartphone, tablet, laptop, or gaming console. A single home automation system may operate door locks, televisions, thermostats, monitors, cameras, lights, and even appliances like the refrigerator. Our smart home system will have access to some of the most useful and versatile features like a "**Fire Prevention System"**, "**Smart Temperature Control System"** and "**An Automated Doorbell System"**. This report will demonstrate the overall architecture and design concept of the systems used in our program and their specifications. The implementation of the system has also been discussed in this report along with limitations of our system. Finally the relevant resources that were used by our team in order to build this system and a user's guide on how our system operates has been described.

Conceptual Design

As a team, our primary focus has been to develop a smart home system that focuses on extending various sections of a home's existing functionality. Our smart home includes a fire prevention system, a home temperature controlling system, and an automated doorbell operator. Throughout the home system, the three individual IOT devices are associated together through the use of Edge devices that are interconnected on a cloud server. From the cloud server, the systems are respectively able to transfer their individual data to the primary web application through a singular database that holds each system's information. As a result, the web application is then able to interpret and display the data of each system on the website and control the status and functionality of the IOT devices. An overall architecture of how the systems used in our IoT solution interacts and associates with one another has been demonstrated by the diagram shown in Figure-1:

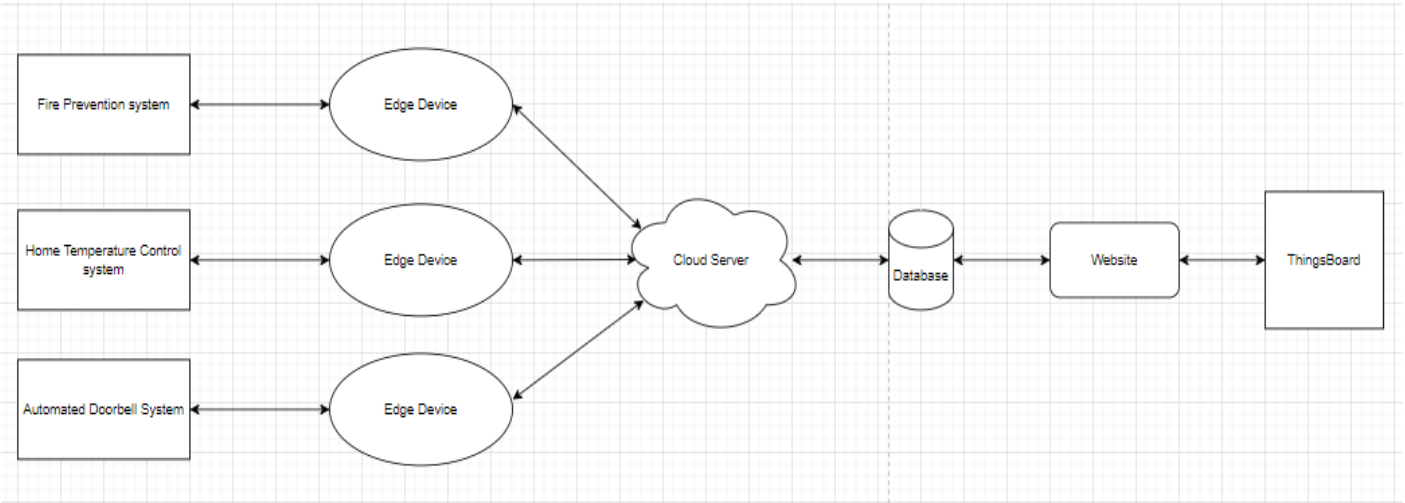


Figure-1: Smart Home IoT Architecture

Fire Prevention System Conceptual Design: Shown below in Figure-2 is a physical conceptual diagram of the home Fire Prevention System, conveying the specific wiring and placements of components, whilst outlining the digital and analogue pins that were implemented in order to operate the system.

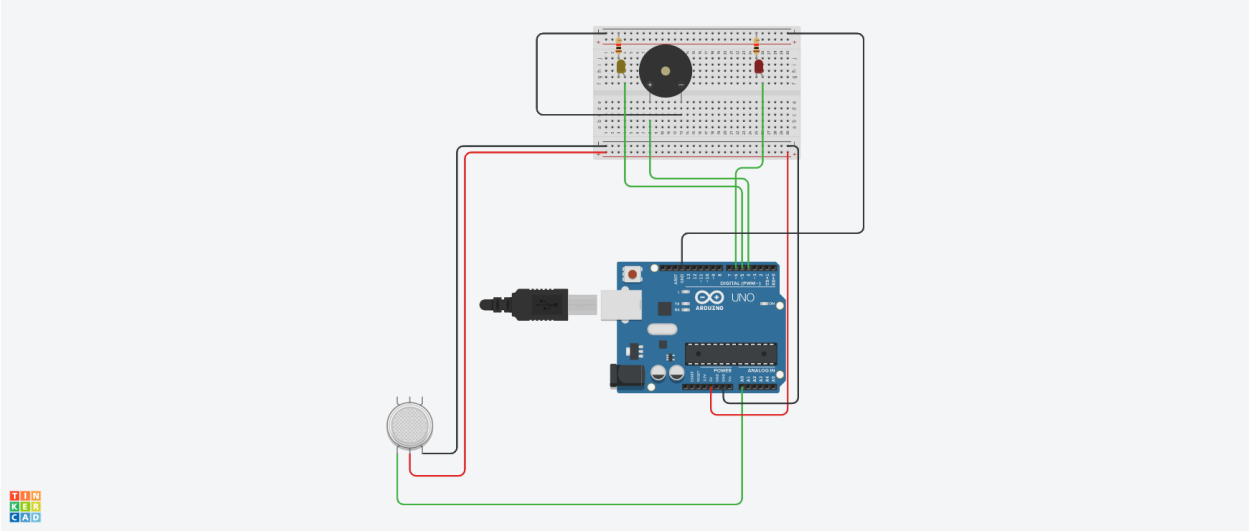


Figure-2: Fire Prevention System Diagram

Smart Temperature Control System Conceptual Design: Figure-3 shows the conceptual diagram for the temperature control system, which displays all the wirings and connections of the entire system. The sensor will send a current measurement of the surrounding temperature and if it goes above 40°C, the fan will work.

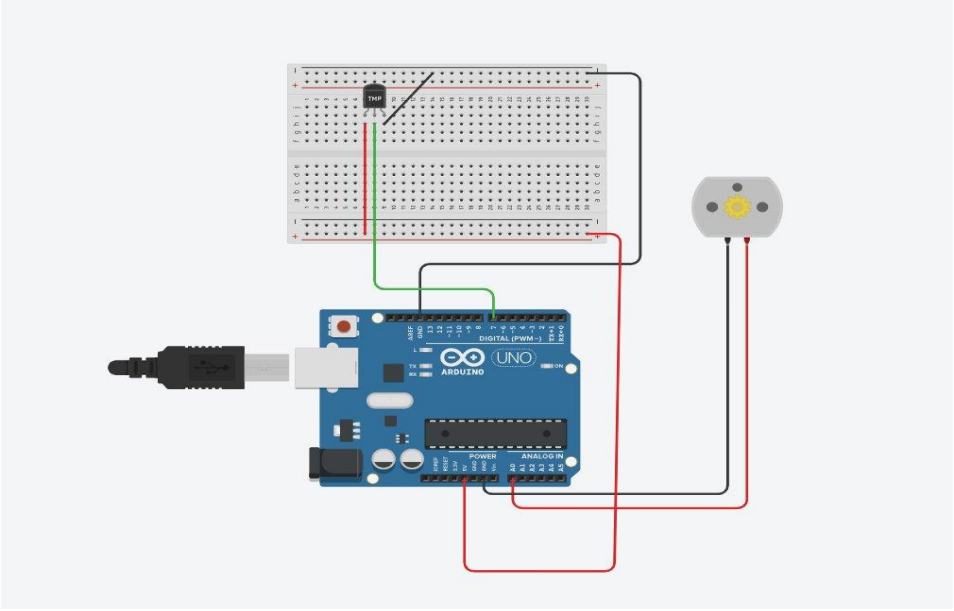


Figure-3: Smart Temperature Control System

Automated Doorbell System Conceptual Design: Represented below in Figure-4 is a detailed conceptual diagram of the Automated Doorbell IOT device that makes use of an ultrasonic sensor to simulate the use of a proximity sensor, a Buzzer to represent the home doorbell and an LED to signify the activity of the doorbell (the doorbell is ringing when the LED is activated, and it is not ringing when the LED is inactive).

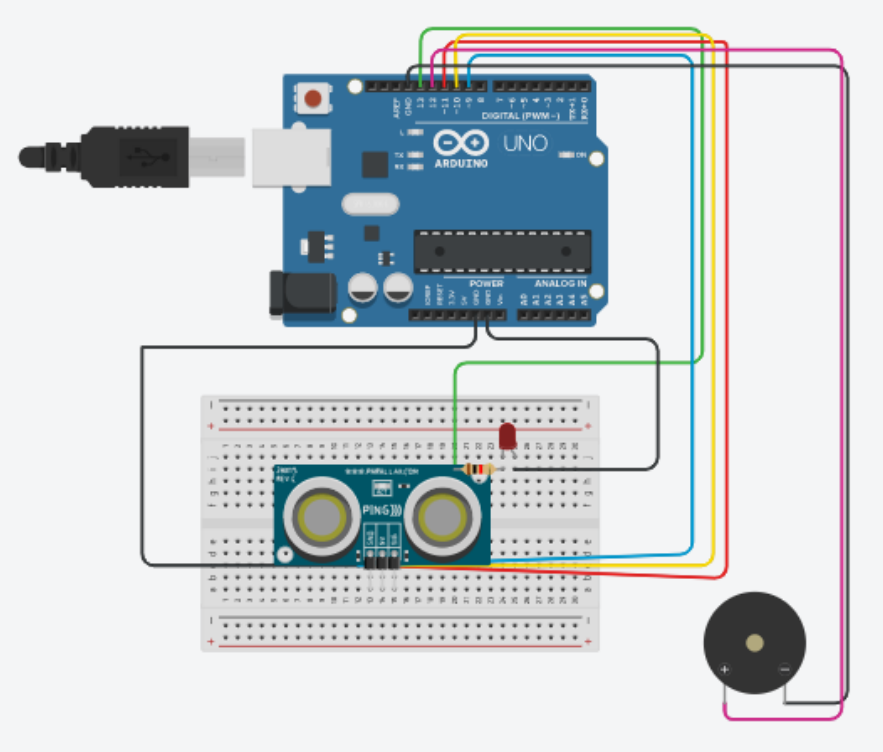


Figure-4: Fire Prevention System Diagram

Task Breakdowns

No.	Abdul Hamid Mahi	Jason Stanley	Valentin Gurzau
1	Designed and connected the physical components like the MQ-2 gas sensor and other two led's (simulating the functionality of an actuator) with the Arduino UNO board.	Responsible for designing and developing the smart temperature control system.	Responsible for designing and developing the smart home doorbell system.
2	Setting up the serial connection between the edge server and the arduino and ensuring that sensor data is being effectively sent.	Setting up MQTT on edge device & cloud	Setting up MQTT on the edge device & cloud.
3	Configuring MQTT protocol on edge device and & Cloud.	Setting up a database to store sensor values, setting up a webserver to display the latest database value, along with Thingsboard.	Collaboratively developing the website and database of the project.
4	Writing the report on the functionality of the Fire Prevention sytem and making relevant diagrams.	Writing report and videos	Developing the report and video of the overall project.

Implementation

Individual IoT Implementations

1. Smart Temperature Control System

The main function of the system is to detect the temperature of its surroundings, enabled by the temperature sensor module. However, if the surrounding temperature exceeds the value of 40°C, there's a DC Motor that will start to operate, which if compared to a real-life situation, will simulate an air conditioning system working.

2. Fire Prevention system

Whenever there is smoke above the concentration of 300ppm the MQ-2 smoke sensor would pick it up and activate the 2 LED lights which are acting as our actuators for opening the door and window. Both the actuators are opened at once when there is smoke above the threshold level and along with the activation of the actuators a Buzzer also goes off. The information gathered from the sensors would be sent via the Arduino UNO processor board to the raspberry-pi edge server made in VirtualBox. This information is then uploaded to the cloud via the MQTT protocol and is stored in a created database that is able to be displayed live within the web application.

3. Automated Doorbell System

The automated doorbell system operates through the use of a proximity sensor, a doorbell and a visual status display, which are each respectively simulated for the purpose of this assignment through the use of an ultrasonic sensor, a buzzer and an LED display. The system operates by automatically ringing the smart home's doorbell (simulated by the buzzer in this instance) once an individual's movement is noticed by the proximity sensor within 30cm of the door's vicinity (simulated by the ultrasonic sensor), and further notifies the home-owner of the visitor through a visual status display in the home (that is simulated by the LED in this case). The data gained by the smart home system is then transferred into a database stored on the cloud, that can thus be further manipulated by the web application.

Sensing Systems

1. **Temperature Sensor (*XC4494*)**

   Used to measure the current surrounding temperature in Celsius (°C)

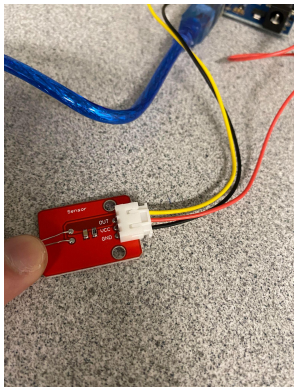


Fig- 5: Snap of temp sensor used in the project

Temperature Sensor Module for Arduino Projects

Provides simple way to measure temperature

This module provides a simple way to measure temperature. The module outputs an analog voltage that varies directly with temperature.

- Operating voltage: 5VDC
- 21cm Breakout cable included
- Dimensions: 33(W) x 22(D) x 9(H)mm

[3]

2. **Proximity Sensor:**

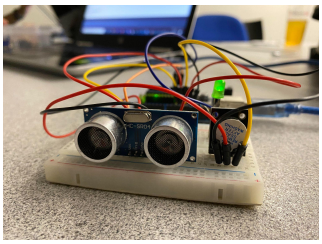


Fig-6: Snap of proximity sensor used in project

An Ultrasonic sensor is used to measure the distance between the sensor itself and a specific object, displayed in Centimeters (CM)

The popular HC-SR04 ultrasonic distance module provides an easy way for your DuinoTECH to measure distances up to 4.5m. Use it to add obstacle avoidance to your next robotics project or mount it on a servo to create your own ultrasonic SONAR!

- Uses just two digital pins
- Operating Voltage: 5V
- Sensor Angle: <15 degrees
- Detection Distance: 2cm – 450cm \
- Sensor IC : HC-SR04
- Dimensions: 45(W) x 20(D) x 13(H)mm

[4]

3. MQ-2 gas sensor

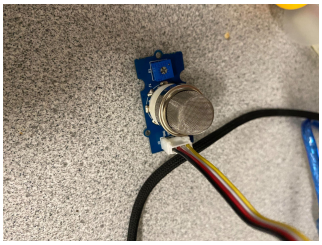


Fig-7: Snap of proximity sensor used in project

Operating voltage	5V
Load resistance	20 KΩ
Heater resistance	33Ω ± 5%
Heating consumption	<800mw
Sensing Resistance	10 KΩ – 60 KΩ
Concentration Scope	200 – 10000ppm

[5]

Actuators

Buzzer:

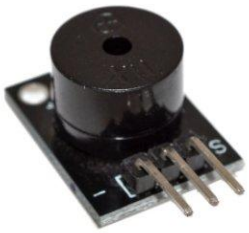


Fig-8: Snap of buzzer used in project

Module Type	Passive
Operating Voltage	3.3-5V
Color	Black
Board size	3.2*1.4cm
Weight	5g

DC Motor:

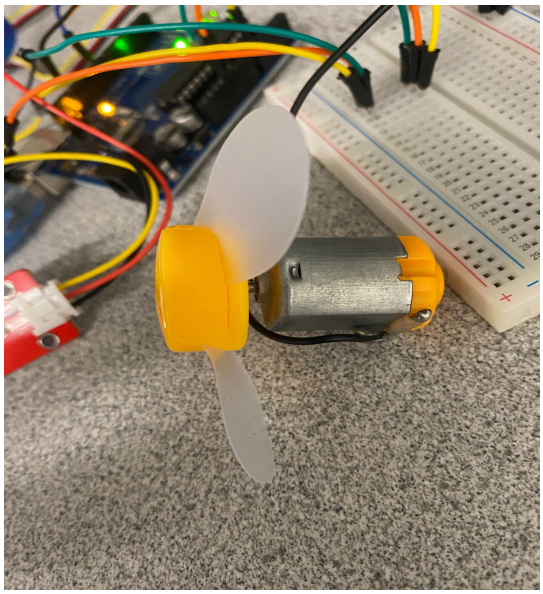


Fig-9: Snap of DC motor and fan used in project

DC motor 6/9V

Item	Specification
Rated Voltage	6V DC
No load speed	12000±15%rpm
No load current	≤280mA
Operating voltage	1.5-6.5V DC
Starting Torque	≥250g.cm(according to ourself developed blade)
starting current	≤5A
Insulation Resistance	above 10Ω between the case and the terminal

Edge Server

Individually throughout our group, all respective members created their own separate edge servers, which were used to receive data from their respective Arduino systems. The edge servers were then used to transfer its specific IoT device data to the cloud service, which would then allow for further use of the data.

Communication Protocols:

In this project, we have used the MQTT protocol as our default communication protocol. MQTT is a standard for IoT messaging. MQTT is an OASIS standard messaging protocol for the Internet of Things (IoT). It is designed as an extremely lightweight publish/subscribe messaging transport that is ideal for connecting remote devices with a small code footprint and minimal network bandwidth. MQTT today is used in a wide variety of industries, such as automotive, manufacturing, telecommunications, oil and gas, etc [2]. We used the MQTT protocol for the following reasons:

- It's lightweight and Efficient
- Reliable Message Delivery
- Bi-Directional Communications
- Scale to Millions of Things

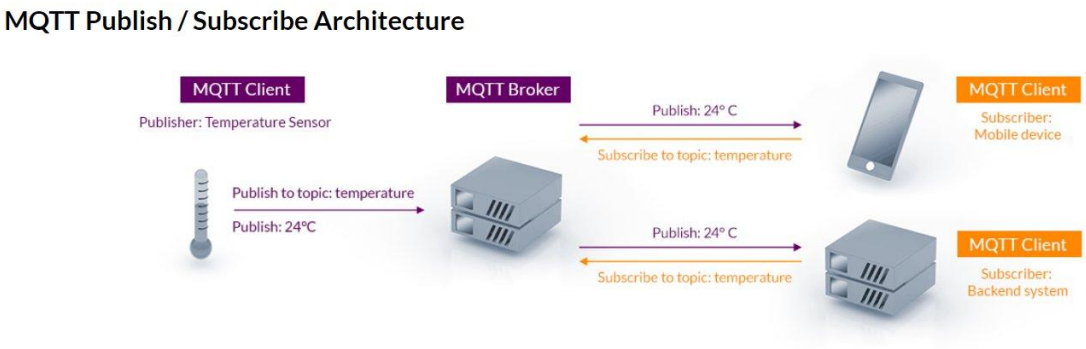
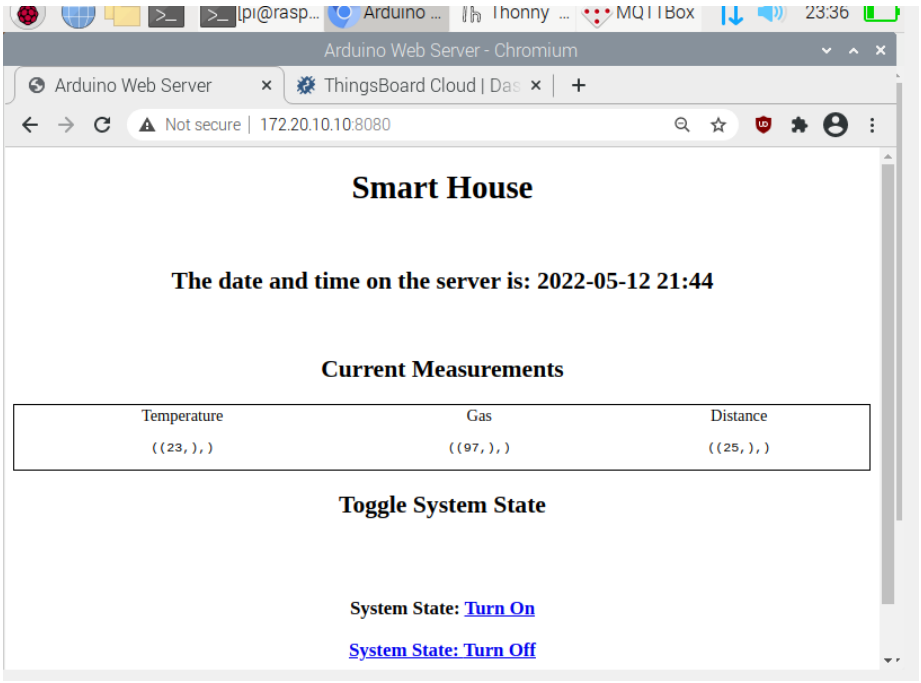


Fig-10: MQTT Architecture [2]

Website Details

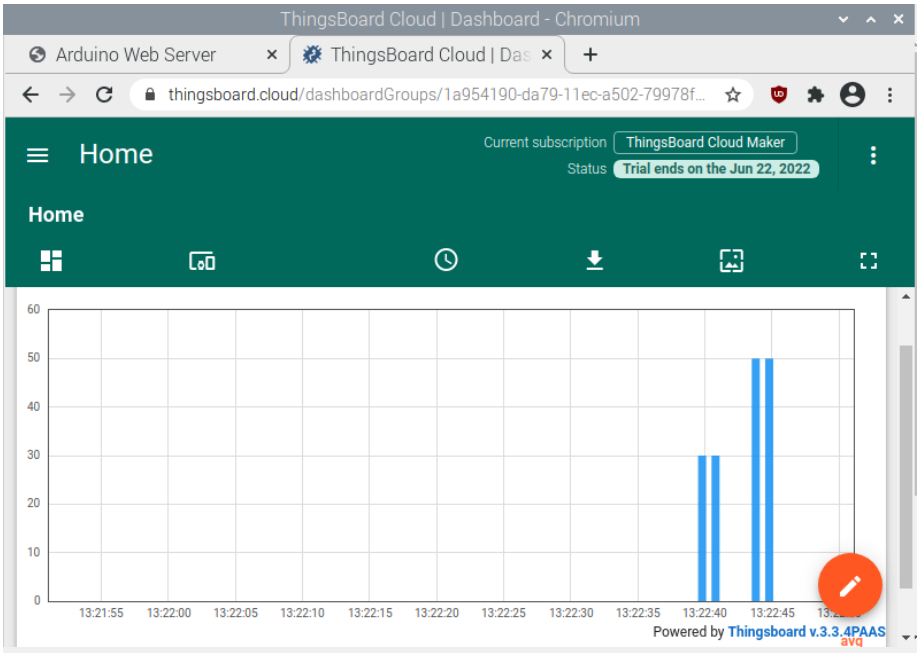
Through the development of the Smart Home System, the use of a web application hosted through the 'Flask' service on a Linux (with Debian) operating virtual machine has been implemented as the primary control point of the system. From the web application, the user is able to view the most recently updated data from the IoT systems in a displayed table, preview the current date and time on the website from a virtual display and access an informative visualization of the data received from the IoT devices. Such is displayed as an example in the following screenshot:

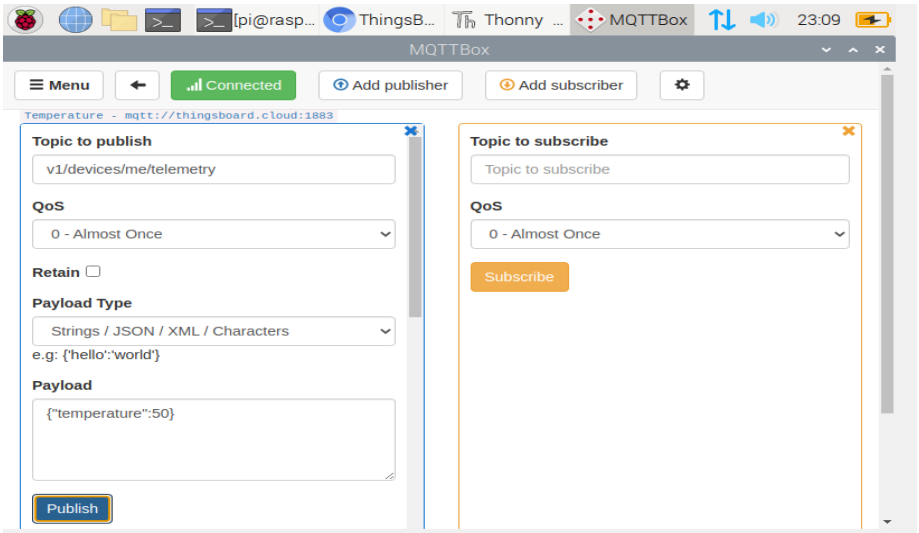


Cloud Computing

When developing the Smart Home system, ThingsBoard had been implemented to display charts that graphically represent the value of the temperature. The ThingsBoard used was via thingsboard.cloud which is openly accessible on the browser and was made inside the cloud server.

Fig-11: Things board temperature graph





User Manual

Requirements:

- All 3 IoT systems need to be able to transfer their respective sensor data into 3 separate edge servers via serial transfer.
- A cloud server consisting of the main database (home_db) that will store all sensor values shall be set up as well as a website that displays the latest database value.
- Tables need to be set up inside the database (homelog), consisting of 4 rows which include the ID, temperature, gas, and distance.
- All 3 edge server needs to be connected to the cloud server via MQTT protocol.
- The database and website need to be connected via FLASK.

To run the system:

1. Connect the Arduino system to a PC (personal computer) that has Raspberry Virtual Machine.
2. Turn on all 3 edge devices as well as the cloud server.
3. Run practical.py (See Appendix) on all 3 edge devices. which is a set of code that allows the edge server to send data to the cloud server via MQTT protocol
4. Run practical.py (See Appendix) on the cloud server, which is a set of code to receive all data from the edge servers, and then store those values inside a MySQL database. It can be accessed using the following commands:
    - Mysql;
    - Use home_db;
    - Select * from homelog;

5. Run web.py to enable web server access, and then type 172.20.10.10:8080 to access the website.

Limitations

Throughout the course of developing the Smart Home system, there have been various limitations that had hindered the potential capabilities of the system, these are listed in the following:

- The thingsboard connectivity in the Smart Home system does not automatically send and display data in the visualizations and requires manual input of data.
- The thingsboard visualization displayed in the web application is only of the Fire Prevention System.
- The operability of the IoT systems is not possible from the web application.
- The database display in the web application displays the most recent data once the webpage has been refreshed only.
- MQ-2 smoke sensor has been programmed to only work for smoke ppm values even though it can pick up LPG and Carbon Monoxide values .

Resources

The Arduino website which has sample codes and libraries:

https://www.arduino.cc/reference/en/libraries/newping/

— ***For calibration of Ultrasonic Sensor***

— ***1.9.4 version***

https://create.arduino.cc/projecthub/ajayrajan69/mq-2-gas-sensor-arduino-c64791/

— ***For calibration of MQ2***

Github links used:
https://github.com/rolan37/MQ-2-Sensor-Interfaced-withArduino/blob/main/Code/MQ2_sensor/MQ2_sensor.ino

https://github.com/rolan37/MQ-2-Sensor-Interfaced-with-Arduino/blob/main/Code/MQ2_sensor/MQ2_sensor.ino

Other Relevant websites:

https://lastminuteengineers.com/mq2-gas-senser-arduino-tutorial/

https://www.jaycar.com.au/arduino-compatible-temperature-and-humidity-sensormodule/p/XC4520

Datasheet used for collecting info:
https://www.elprocus.com/an-introduction-to-mq2-gas-sensor/

YouTube videos used for :

https://www.youtube.com/watch?v=MojSo7OtF9w

https://www.youtube.com/watch?v=jbUgHKAMcvo&t=230s

https://www.youtube.com/watch?v=OogIdLc9uYc&t=98s

https://www.youtube.com/watch?v=HynLoCtUVtU

https://www.youtube.com/watch?v=1J1lYluQEwo

https://www.youtube.com/watch?v=ZejQOX69K5M

Swinburne Notes Used:

https://swinburne.instructure.com/courses/40781/files/16519588?module_item_id=2520286

https://swinburne.instructure.com/courses/40781/files/16519402?module_item_id=2520276

https://swinburne.instructure.com/courses/40781/files/18450863?module_item_id=2827070

https://swinburne.instructure.com/courses/40781/files/16519606?module_item_id=2520292

Appendix

AC.uno


```
int fan = 9;

void setup() {
  // Begin serial communication at 9600 baud rate
  Serial.begin(9600);
  pinMode(fan, OUTPUT);
}

void loop() {
  int reading = analogRead(A0);

  // Convert that reading into voltage
  float voltage = 5.0 * reading / 1024 ;

  // Convert the voltage into the temperature in Celsius
  float temperatureC = (voltage * 100) - 210;
  Serial.println(temperatureC);

  delay(1000); // wait a second between readings

  if (temperatureC > 40) {
    analogWrite(fan,255);
    delay(1000);
    }
   else{
    analogWrite(fan,0);
   }
}
```

FireSafety.uno

FireSafety.ino

```
#include <dht11.h>


//global varaibles
dht11 DHT;
int MQpin = A0;
int DHTpin = 8;
int lpg, co, smoke;
int buzzer = 4;
int window = 5;
int door = 6;
int active = HIGH;
int inactive = LOW;
int smokeThreshold = 400;

void setup(){
  Serial.begin(9600);
  pinMode(buzzer, OUTPUT);
  pinMode(window,OUTPUT);
  pinMode(door,OUTPUT);
  pinMode(DHTpin, OUTPUT);

}


void loop()

{

  float smokeValues= analogRead(A0);
```

FireSafety.ino

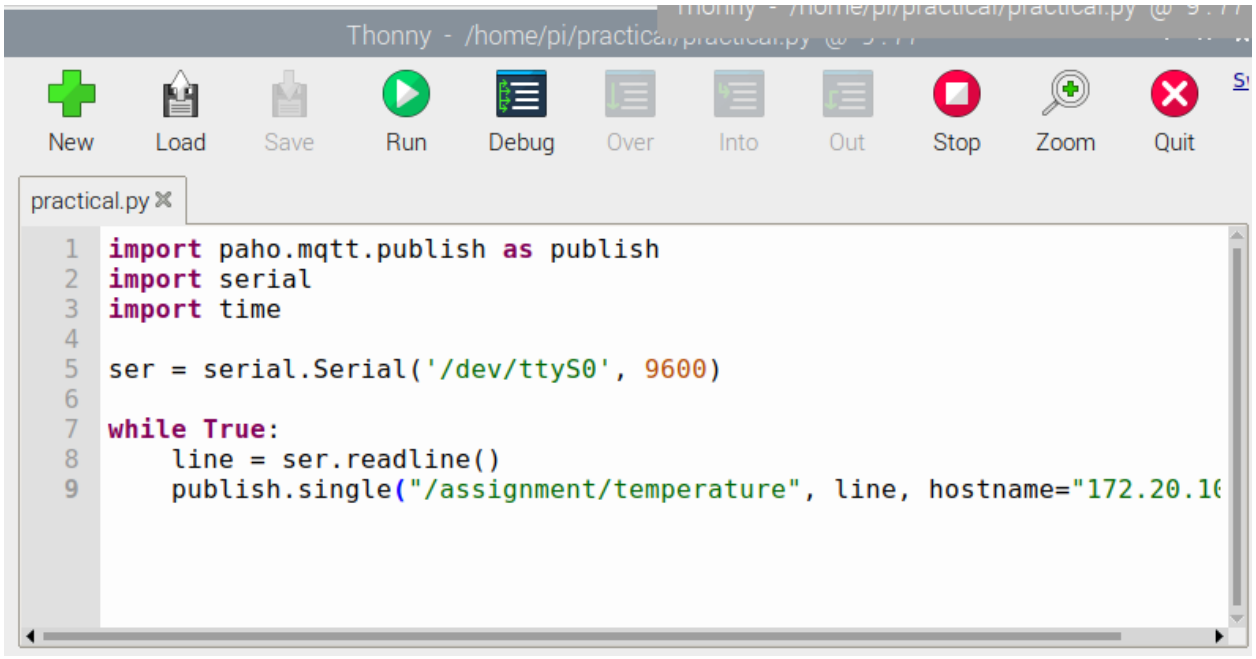
```
  float smokeValues= analogRead(A0);

  Serial.println(smokeValues);

  delay(100);

 if (Serial.available()>0)
 {
   //Read serial input
   int value = Serial.read();

   if (value == '1')
   {
     digitalWrite(window,active);
   }

   else if (value == '2')
   {
     digitalWrite(window,inactive);
   }

   else if (value == '3')
   {
     digitalWrite(door,active);
   }

   else if (value == '4')
   {
     digitalWrite(door,inactive);
   }
```

Doorbell.uno

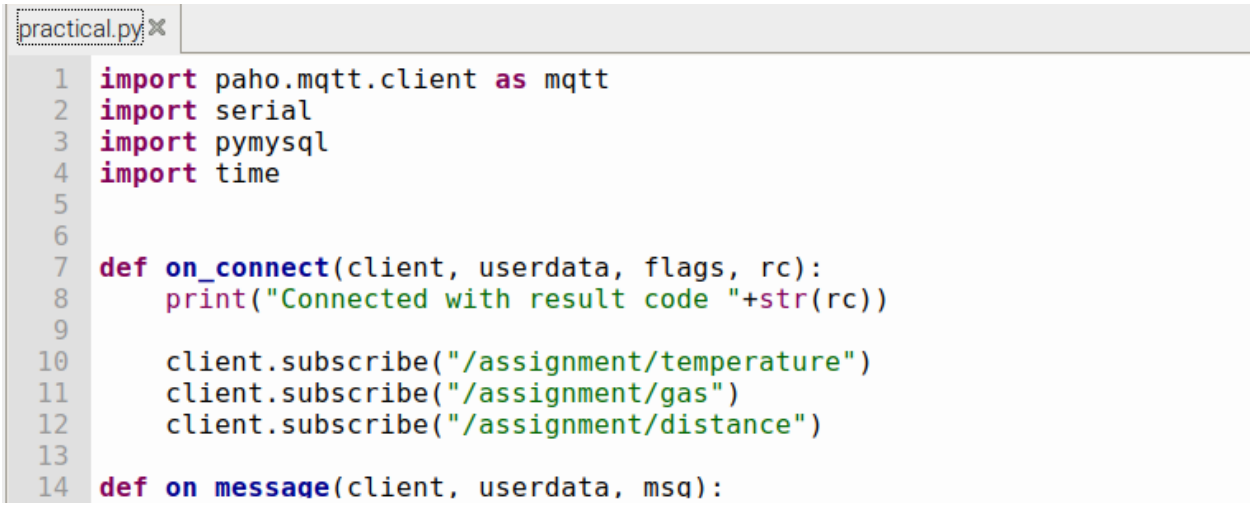
Doorbell §

```
#include <Ethernet.h>
#include <NewPing.h>

const int buzzer = 12;
const int ledPin = 13;

NewPing sonar (10, 11);

void setup() {
  // put your setup code here, to run once:
  pinMode(buzzer, OUTPUT);
  pinMode(ledPin, OUTPUT);

  Serial.begin(9600);
  delay(50);
}

void loop() {
  // put your main code here, to run repeatedly:

  int distance = sonar.ping_cm();

  Serial.println(distance);
  delay(50);

  if (distance <= 30 && distance > 1) {
    digitalWrite(buzzer, HIGH);

    digitalWrite(ledPin, HIGH);
    delay(distance*5);
  }
  else {

    digitalWrite(buzzer, LOW);
    digitalWrite(ledPin, LOW);
    delay(distance*5);
  }
}
```

Practical.py (MQTT on Edge Server)



Practical.py (MQTT on Cloud to store database)



practical.py

```
14 def on_message(client, userdata, msg):
15     global global_var
16     print(msg.topic+" "+str(msg.payload))
17     data = msg.payload
18
19     if msg.topic == "/assignment/temperature":
20
21         dbConn = pymysql.connect("localhost", "pi","","home_db") or die
22
23         with dbConn:
24             cursor = dbConn.cursor()
25             cursor.execute("INSERT INTO homelog (temperature) VALUES(%s
26             dbConn.commit
27             cursor.close()
```

practical.py

```
27             cursor.close()
28
29     if msg.topic == "/assignment/gas":
30
31         dbConn = pymysql.connect("localhost", "pi","","home_db") or die
32
33         with dbConn:
34             cursor = dbConn.cursor()
35             cursor.execute("INSERT INTO homelog (gas) VALUES(%s)"%(data
36             dbConn.commit
37             cursor.close()
38
39     if msg.topic == "/assignment/distance":
40
```

practical.py

```
38
39     if msg.topic == "/assignment/distance":
40
41         dbConn = pymysql.connect("localhost", "pi","","home_db") or die
42
43         with dbConn:
44             cursor = dbConn.cursor()
45             cursor.execute("INSERT INTO homelog (distance) VALUES(%s)"%
46             dbConn.commit
47             cursor.close()
48
49
50 client = mqtt.Client()
51 client.on_connect = on_connect
```

practical.py

```
43          with dbConn:
44              cursor = dbConn.cursor()
45              cursor.execute("INSERT INTO homelog (distance) VALUES(%s)"%
46              dbConn.commit
47              cursor.close()
48
49
50  client = mqtt.Client()
51  client.on_connect = on_connect
52  client.on_message = on_message
53
54  client.connect("172.20.10.10", 1883, 60)
55
56  client.loop_forever()
```

Web.py (Python file for website)

practical.py | web.py

```
1   import time
2   import datetime
3   import pymysql
4   from flask import Flask, render_template
5   import serial
6   import re
7   import paho.mqtt.client as mqtt
8
9
10  app = Flask(__name__)
11
12  @app.route("/")
13
14  def index():
```

practical.py | web.py

```
14  def index():
15
16      now = datetime.datetime.now()
17      timeString = now.strftime("%Y-%m-%d %H:%M")
18
19      dbconn = pymysql.connect("localhost","pi","","home_db") or die("cou
20
21      with dbconn:
22          cursor = dbconn.cursor()
23          cursor.execute("SELECT temperature FROM homelog WHERE temperatu
24          temp = cursor.fetchall()
25          cursor.execute("SELECT gas FROM homelog WHERE gas IS NOT NULL (
26          gas = cursor.fetchall()
27          cursor.execute("SELECT distance FROM homelog WHERE distance IS
```

practical.py web.py

```
        cursor.execute("SELECT gas FROM homelog WHERE gas IS NOT NULL (
        gas = cursor.fetchall()
        cursor.execute("SELECT distance FROM homelog WHERE distance IS
        distance = cursor.fetchall()

    templateData = {
        'time': timeString
        }

    return render_template('index.html',temp = temp, gas = gas, distanc


if __name__ == '__main__':
```

```
    return render_template('index.html',temp = temp, gas = gas, distanc


if __name__ == '__main__':
    app.run(host='0.0.0.0', port = 8080, debug = True)
```

Index.html

index.html

```
<!DOCTYPE html>
    <head>
        <title>Arduino Web Server</title>
    </head>
    <body style="text-align:center;">
        <h1>Smart House</h1>
        <br/>
        <h2>The date and time on the server is: {{time}}</h2>
        <br/>
        <h2>Current Measurements</h2>
        <table style="width:100%; border: 1px solid;">
        <tr>
            <td>Temperature</td>
            <td>Gas</td>
            <td>Distance</td>
        </tr>
        <tr>
            <td><pre>{{ temp }}</pre></td>
```

index.html

```
        </tr>
        <tr>
            <td><pre>{{ temp }}</pre></td>
            <td><pre>{{ gas }}</pre></td>
            <td><pre>{{ distance }}</pre></td>
        </tr>
        </table>

    <h2> Toggle System State </h2>
    {% for Pins in pins %}
       {% if pins[Pins].state ==1 %}

       Currently, the suppression system is <strong>on</strong>

       {% else %}

       Currently, the suppression system is <strong>off</strong>
```

```
        Currently, the suppression system is <strong>off</strong>

        {% endif %}
    {% endfor %}

    <br></br>
    <h3> System State: <a href="/action1">Turn On </h3>
    <h3> System State: <a href="/action2">Turn Off </h3>
    <br></br>
    </body>
</html>
```

Latest Database Entry:

```
| 117 |          NULL | NULL |        38 |
| 118 |          NULL | NULL |        19 |
| 119 |          NULL | NULL |         6 |
| 120 |            23 | NULL |      NULL |
| 121 |          NULL | NULL |         7 |
| 122 |          NULL | NULL |        12 |
| 123 |          NULL |   96 |      NULL |
| 124 |          NULL | NULL |        24 |
| 125 |          NULL | NULL |        19 |
| 126 |          NULL | NULL |        12 |
| 127 |          NULL | NULL |         8 |
| 128 |            23 | NULL |      NULL |
| 129 |          NULL | NULL |        13 |
| 130 |          NULL | NULL |        24 |
| 131 |          NULL |   97 |      NULL |
| 132 |          NULL | NULL |        25 |
| 133 |            23 | NULL |      NULL |
| 134 |            23 | NULL |      NULL |
| 135 |            23 | NULL |      NULL |
| 136 |            23 | NULL |      NULL |
+-----+---------------+------+-----------+
77 rows in set (0.000 sec)
```

References

[1]
“Buzzer Arduino Laser Module 3 Pin Outlet 3.3-5V Electronic Passive Alarm Module,” *Diyelectronicskit.com*, 2013. https://www.diyelectronicskit.com/sale-11478935-buzzer-arduino-laser-module-3-pin-outlet-3-3-5v-electronic-passive-alarm-module.html (accessed May 23, 2022).

[2]
“MQTT - The Standard for IoT Messaging,” *Mqtt.org*, 2022. https://mqtt.org/ (accessed May 23, 2022).

[3]

“Temperature Sensor Module Arduino Compatible | Jaycar Electronics,” *Jaycar.com.au*, 2022. https://www.jaycar.com.au/temperature-sensor-module-arduino-compatible/p/XC4494 (accessed May 23, 2022).

[4]

“Arduino Compatible Dual Ultrasonic Sensor Module. | Jaycar Electronics,” *Jaycar.com.au*, 2022. https://www.jaycar.com.au/arduino-compatible-dual-ultrasonic-sensor-module/p/XC4442 (accessed May 23, 2022).

[5]

labay11, “MQ-2-sensor-library/arduino_sample.ino at master · labay11/MQ-2-sensorlibrary,” GitHub, Feb. 21, 2021. https://github.com/labay11/MQ-2-sensorlibrary/blob/master/arduino_sample/arduino_sample.ino (accessed Apr. 26

[6]

Z-HUT, “Arduino Sensors: Analog temperature sensor module (thermistor),” *YouTube*. May 09, 2019. Accessed: May 23, 2022. [YouTube Video]. Available: https://www.youtube.com/watch?v=VLyOcUf8I_U

[7]

“Buzzer Arduino Laser Module 3 Pin Outlet 3.3-5V Electronic Passive Alarm Module,” *Diyelectronicskit.com*, 2013. https://www.diyelectronicskit.com/sale-11478935-buzzer-arduino-laser-module-3-pin-outlet-3-3-5v-electronic-passive-alarm-module.html (accessed May 23, 2022).

[8]

“DC motor 6/9V.” [Online]. Available: https://www.arduino.cc/documents/datasheets/DCmotor6_9V.pdf

[9]

How To Mechatronics, “Ultrasonic Sensor HC-SR04 and Arduino Tutorial,” *YouTube*. Jul. 26, 2015. Accessed: May 23, 2022. [YouTube Video]. Available: https://www.youtube.com/watch?v=ZejQOX69K5M

[10]

How To Mechatronics, “Ultrasonic Sensor HC-SR04 and Arduino Tutorial,” *YouTube*. Jul. 26, 2015. Accessed: May 23, 2022. [YouTube Video]. Available: https://www.youtube.com/watch?v=ZejQOX69K5M